

SCR2043 OPERATING SYSTEMS

Name : CHERYL CHEONG KAH VOON
Student ID : A23CS0060
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

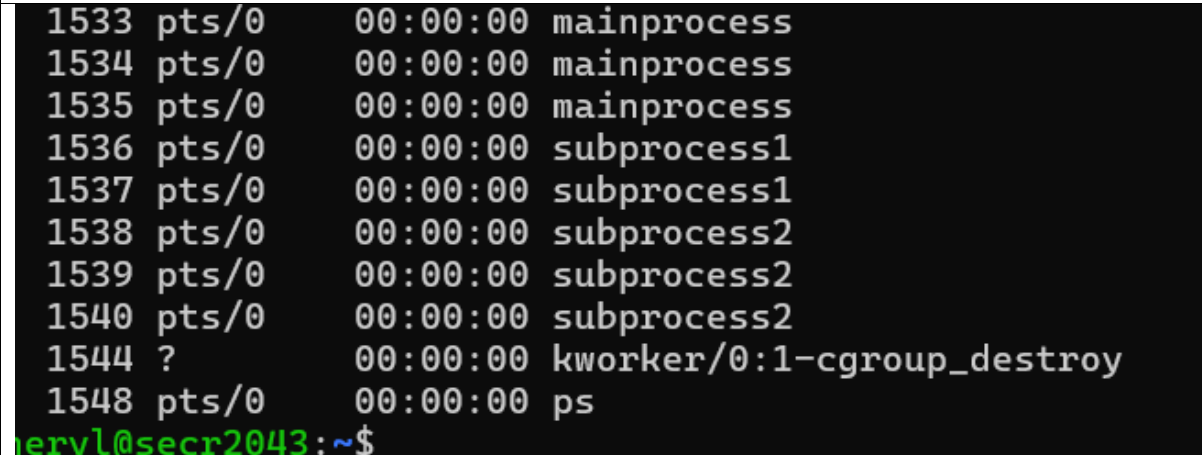
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

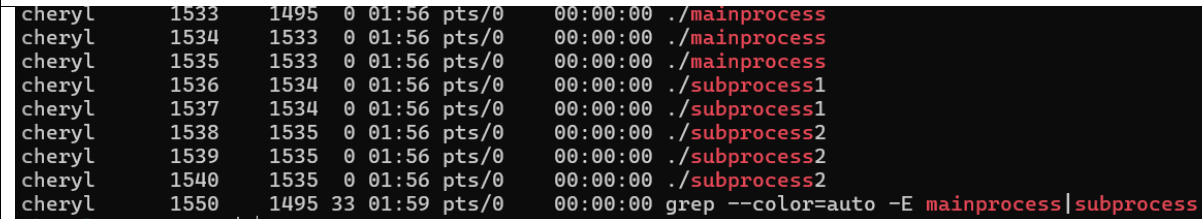
Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command	
<code>ps -e</code>	
Output	
	

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command	
<code>ps -ef grep -E 'mainprocess subprocess'</code>	
Output	
	

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command	
<code>ps -ef grep -E 'subprocess'</code>	
Output	
<pre>cheryl@secr2043:~\$ ps -ef grep -E 'subprocess' cheryl 1536 1534 0 01:56 pts/0 00:00:00 ./subprocess1 cheryl 1537 1534 0 01:56 pts/0 00:00:00 ./subprocess1 cheryl 1538 1535 0 01:56 pts/0 00:00:00 ./subprocess2 cheryl 1539 1535 0 01:56 pts/0 00:00:00 ./subprocess2 cheryl 1540 1535 0 01:56 pts/0 00:00:00 ./subprocess2 cheryl 1562 1495 50 02:06 pts/0 00:00:00 grep --color=auto -E subprocess</pre>	

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command	
<code>ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess'</code>	
Output	
<pre>cheryl@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess' 1536 cheryl 0.0 0.1 ./subprocess1 1537 cheryl 0.0 0.1 ./subprocess1 1538 cheryl 0.0 0.1 ./subprocess2 1539 cheryl 0.0 0.1 ./subprocess2 1540 cheryl 0.0 0.1 ./subprocess2 1645 cheryl 33.3 0.1 grep --color=auto -E subprocess</pre>	

Question 5

Building on the `ps` command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess'</code>


```

cheryl@secre2043:~$ pstree -p -t
systemd(1)-+-ModemManager(702)-+-{gdbus}(720)
                                   |-{gmain}(715)
                                   `-{pool-spawner}(717)
    |-cron(757)
    |-dbus-daemon(644)
    |-fwupd(1581)-+-{GUsbEventThread}(1584)
                  |-{gdbus}(1587)
                  |-{gmain}(1582)
                  |-{libusb_event}(1583)
                  `-{pool-spawner}(1585)
    |-login(778)---bash(1480)
    |-multipathd(364)-+-{multipathd}(374)
                      |-{multipathd}(375)
                      |-{multipathd}(376)
                      |-{multipathd}(377)
                      |-{multipathd}(378)
                      `-{multipathd}(379)
    |-polkitd(648)-+-{gdbus}(688)
                  |-{gmain}(686)
                  `-{pool-spawner}(687)
    |-rsyslogd(675)-+-{in:imklog}(708)
                   |-{in:imuxsock}(707)
                   `-{rs:main Q:Reg}(709)
    |-sshd(1491)---sshd(1492)---sshd(1494)---bash(1495)-+-+
                                                         `--+
    |-systemd(1446)---(sd-pam)(1453)
    |-systemd-journal(305)
    |-systemd-logind(655)
    |-systemd-network(515)
    |-systemd-resolve(520)
    |-systemd-timesyn(521)---{sd-resolve}(552)
    |-systemd-udev(370)
    |-udisksd(656)-+-{cleanup}(713)
                   |-{gdbus}(682)
                   |-{gmain}(678)
                   |-{pool-spawner}(680)
                   `-{probing-thread}(706)
    `--unattended-upgr(710)---{gmain}(777)

```

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>sudo renice -n 10 -p \$(pgrep subprocess1)</code>
Output

```
cheryl@secr2043:~$ sudo renice -n 10 -p $(pgrep subprocess1)
1536 (process ID) old priority 0, new priority 10
1537 (process ID) old priority 0, new priority 10
```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output
<pre>cheryl@secr2043:~\$ killall -15 mainprocess Main process (ID: 1533) received signal: 15. Terminating... Main process (ID: 1535) received signal: 15. Terminating... [1]+ Done ./mainprocess Main process (ID: 1534) received signal: 15. Terminating... cheryl@secr2043:~\$</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
nano example.c
gcc example.c -o example
gcc./example
```

```
example.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
void child_process() {
printf("Child process with PID: %d\n", getpid());
int sleep_time = rand() % 5 + 1;
printf("Child process sleeping for %d seconds\n", sleep_time);
sleep(sleep_time);
printf("Child process exiting\n");
}
int main() {
printf("Parent process with PID: %d\n", getpid());
// Fork a child process
pid_t pid = fork();
if (pid == 0) {
// This is the child process
child_process();
exit(0);
} else if (pid > 0) {
// This is the parent process
printf("Parent process waiting for child process to finish\n");
// Wait for the child process to finish
wait(NULL);
printf("Parent process exiting\n");
} else {
// Error occurred while forking
perror("fork");
return 1;
}
return 0;
}
```

Output:

```

GNU nano 7.2                                example.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
void child_process() {
printf("Child process with PID: %d\n", getpid());
int sleep_time = rand() % 5 + 1;
printf("Child process sleeping for %d seconds\n", sleep_time);
sleep(sleep_time);
printf("Child process exiting\n");
}
int main() {
printf("Parent process with PID: %d\n", getpid());
// Fork a child process
pid_t pid = fork();
if (pid == 0) {
// This is the child process
child_process();
exit(0);
} else if (pid > 0) {
// This is the parent process
printf("Parent process waiting for child process to finish\n");
// Wait for the child process to finish
wait(NULL);
printf("Parent process exiting\n");
} else {
// Error occurred while forking
perror("fork");
return 1;
}
return 0;
}

```

```

cheryl@secr2043:~$ nano example.c
cheryl@secr2043:~$ gcc example.c -o example
cheryl@secr2043:~$ ./example
Parent process with PID: 1740
Parent process waiting for child process to finish
Child process with PID: 1741
Child process sleeping for 4 seconds

```